

```
import json
import os
from datetime import date

FILE_NAME = "expenses.json"

def load_expenses():
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, "r") as f:
                return json.load(f)
        except json.JSONDecodeError:
            print("Warning: data file was
corrupted. Starting with empty records.")
            return []
    return []

def save_expenses(data):
    with open(FILE_NAME, "w") as f:
        json.dump(data, f, indent=4)

def get_valid_amount():
    while True:
        amount = input("Enter amount: ")
        try:
            amt = float(amount)
            if amt <= 0:
                print("Amount must be
greater than 0.")
                continue
```

```
        return amt
    except ValueError:
        print("Please enter a valid
number (e.g., 250 or 99.50).")

def add_expense(data):
    amount = get_valid_amount()
    category = input("Enter category
(Food/Travel/Bills/Other): ")
    note = input("Enter note: ")
    data.append({"date": str(date.today()),
"amount": amount, "category": category,
"note": note})
    save_expenses(data)
    print("Expense added!")

def view_expenses(data):
    if not data:
        print("No expenses recorded.")
        return
    for i, e in enumerate(data, 1):
        print(f"{i}. {e['date']} | Rs.
{e['amount']} | {e['category']} |
{e['note']}")

def total_by_category(data):
    if not data:
        print("No expenses recorded.")
        return
    totals = {}
    for e in data:
        totals[e['category']] =
totals.get(e['category'], 0) + e['amount']
    for cat, amt in totals.items():
```

```

        print(f"{cat}: Rs.{amt}")

def monthly_report(data):
    if not data:
        print("No expenses recorded.")
        return
    month = input("Enter month (YYYY-MM),
e.g. 2026-07: ")
    monthly_total = 0
    found = False
    for e in data:
        if e['date'].startswith(month):
            print(f"{e['date']} | Rs.
{e['amount']} | {e['category']} |
{e['note']}")
            monthly_total += e['amount']
            found = True
    if found:
        print(f"Total for {month}: Rs.
{monthly_total}")
    else:
        print("No expenses found for that
month.")

def update_expense(data):
    view_expenses(data)
    idx_input = input("Enter number to
update: ")
    if not idx_input.isdigit():
        print("Invalid entry.")
        return
    idx = int(idx_input) - 1
    if 0 <= idx < len(data):
        data[idx]['amount'] =

```

```

get_valid_amount()
    data[idx]['category'] =
input("Enter new category: ")
    data[idx]['note'] = input("Enter
new note: ")
    save_expenses(data)
    print("Updated!")
else:
    print("Invalid entry.")

def delete_expense(data):
    view_expenses(data)
    idx_input = input("Enter number to
delete: ")
    if not idx_input.isdigit():
        print("Invalid entry.")
        return
    idx = int(idx_input) - 1
    if 0 <= idx < len(data):
        data.pop(idx)
        save_expenses(data)
        print("Deleted!")
    else:
        print("Invalid entry.")

def main():
    data = load_expenses()
    while True:
        print("\n1. Add 2. View 3. Totals
by Category 4. Update 5. Delete 6.
Monthly Report 7. Exit")
        choice = input("Enter choice: ")
        if choice == '1':
            add_expense(data)

```

```
elif choice == '2':  
    view_expenses(data)  
elif choice == '3':  
    total_by_category(data)  
elif choice == '4':  
    update_expense(data)  
elif choice == '5':  
    delete_expense(data)  
elif choice == '6':  
    monthly_report(data)  
elif choice == '7':  
    break  
else:  
    print("Invalid choice.")  
  
if __name__ == "__main__":  
    main()
```